Open in app ↗ Sign up Sign In

Published in Towards Data Science

Thomas Albin Follow

Jun 30, 2020 · 8 min read · ✨ · Listen

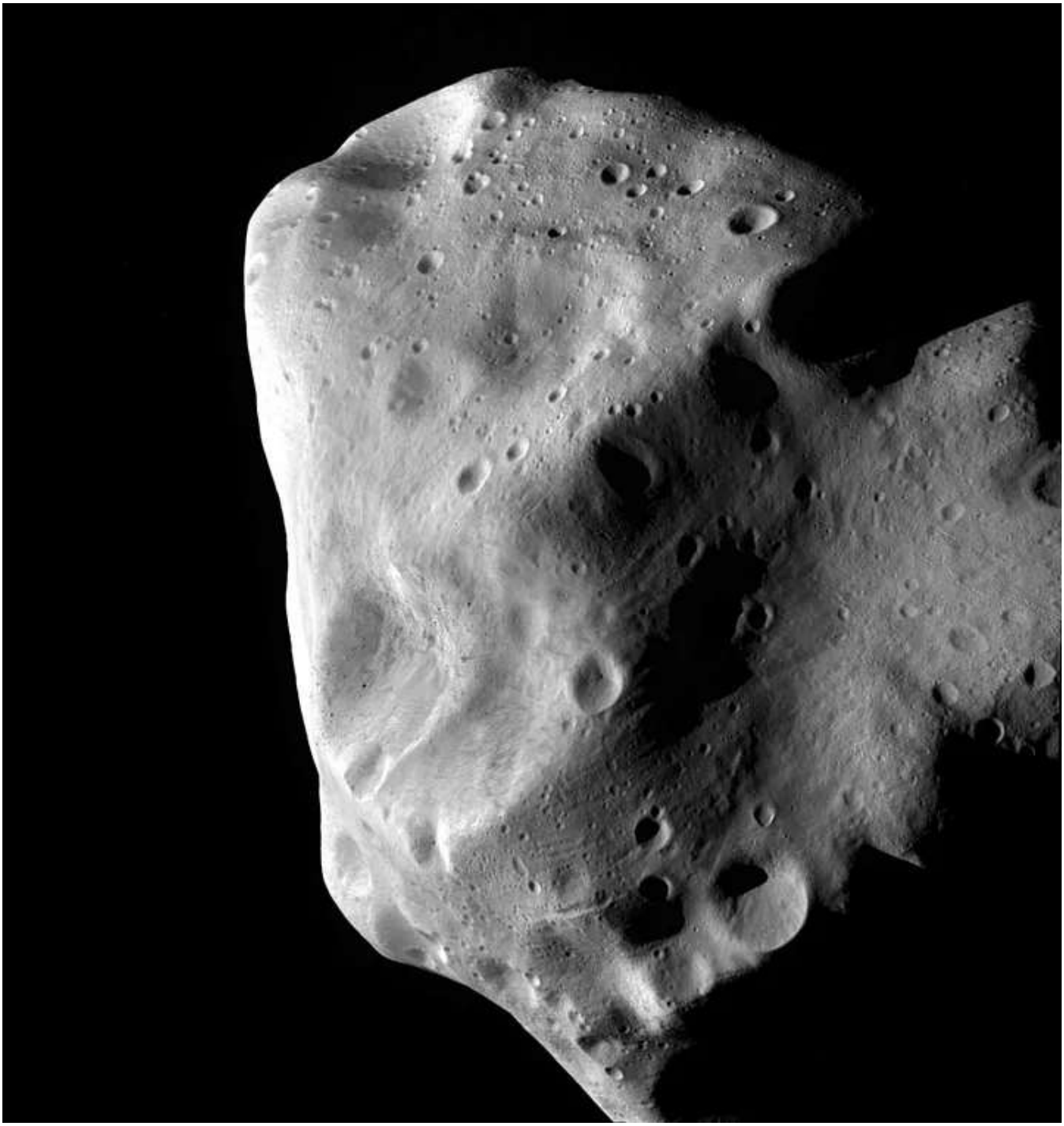
Save

SPACE SCIENCE WITH PYTHON

Space Science with Python — Uncertain Movements of an Asteroid

Part 17 of the tutorial series. Today (30th June) is Asteroid Day and we will start to deep dive into the science of asteroids.

21



(21) Lutetia. A large asteroid (largest diameter: around 100 km) in the main belt between Mars and Jupiter. The image was taken by ESA's Rosetta spacecraft during a flyby in the year 2010. Credit: ESA 2010 MPS for OSIRIS Team MPS/UPD/LAM/IAA/RSSD/INTA/UPM/DASP/IDA; License: Creative Commons Attribution 4.0 International License

Tunguska 1908

It's the 30th June 1908. A huge explosion devastated in a short period of time hundreds of square-kilometres in the Russian region of Tunguska. Millions of trees were bend and burnt down in this Siberian region. For the researchers and people who live there, this so-called *Tunguska Event* was a mystery of unknown cause.

This coloured photo was taken 21 years after the Tunguska Event. The logs point away from a theoretical shock wave direction (right to left). Image from Wikipedia; Credit: Vokrug Sveta

What happened? Well, there are several explanations:

- A gas explosion. A large amount of methane or other gases ignited and caused this event.
- Another geophysical explanation: A volcanic-like eruption was mistaken for an explosion and devastated everything around the eruption centre
- An asteroid or comet entered the Earth's atmosphere and exploded on its way to the surface. Similar as the Meteor of Chelyabinsk in the year 2013 that had a diameter of only 20 m

Dash cam footage of the Chelyabinsk meteor in Russia. An explosion, several km above the surface caused a shock wave that destroyed doors, gates and windows. Hundreds of people have been injured, but luckily no one died.

The last option is currently the most accepted theory. One assumes that the diameter of this asteroid was between 30 and 70 metres. It shows that even small celestial objects can have a catastrophic environmental impact on our home planet.

To remind us of cosmic hazards the Asteroid Day was introduced a few years ago. The date: 30th June — The day of the Tunguska Event.

Asteroids passing by

So what can we do about it? Shall we send *Bruce Willis — like* astronauts on heroic missions against these objects like in the movie *Armageddon*? Well, the first step would be, to discover, observe and catalogue these objects! Thanks to modern telescope systems and dedicated asteroid and comet surveys we identified already hundreds of so-called potentially hazardous asteroids. Do not worry though: all known objects are currently on no collision course with our home planet. However, we still need to continue our observation campaigns, since a lot of objects are probably unknown yet, as described in my previous article about comets.

Space Science with Python — Did we observe everything?

Part 11 of the tutorial series is about a Data Scientist's and Space Scientist's nightmare: Statistical Bias!

towardsdatascience.com

So what is the current status for Asteroid Day 2020? Take a quick look at the website spaceweather.com. Well … not today but at least yesterday a 62 m diameter object passed by Earth at a distance of around 3 times the distance between Earth and Moon. Its name: 2020 JX1.

But why should we care about objects that have been discovered, analysed and whose orbits are known? We have the orbital elements and we need no more, right?

Well, first of all, these object are affected by gravitational perturbations. Their orbits change all the time. Second, we cannot observe objects with 100 % accuracy. There are always measurement errors. And these errors affect the orbital elements of an objects.

Today, we will take a look at these errors and see that we always need so-called *follow-up* observations to optimise the prediction of the asteroids' movements, to keep our home planet save from unexpected visitors.

Asteroid 2020 JX1

For the tutorial, we will get the asteroid data from the NASA / JPL Small-Body database system (in a moment), and our already introduced packages *pandas*, *numpy* and *spiceypy* (later we need also *scipy* and *matplotlib*). We load a SPICE kernel meta file in line 7 and extract the gravitational constant times mass of the Sun in line 10 and 11 using the function bodvcd. This value is later used to determine the position of the Sun-orbiting asteroid 2020 JX1.

Now let's get the orbital elements of the asteroid 2020 JX1. The NASA / JPL Small-Body Database provides detailed information of minor objects. There we find the following orbital elements and the corresponding 1-sigma uncertainties:

Orbital Elements at Epoch 2459000.5 (2020-May-31.0) TDB
Reference: JPL 14 (heliocentric ecliptic J2000)

Element	Value	Uncertainty (1-sigma)	Units
e	.2934873932898034	5.9256e-05	
a	1.423940646203508	0.00011672	au
q	1.006032017749843	1.9102e-06	au
i	3.54821709679635	0.00054823	deg
node	274.5788314475524	9.5903e-05	deg
peri	12.81238701655224	0.00014982	deg
M	337.8534765018636	0.0028077	deg
t_p	2459038.680298393594 (2020-Jul-08.18029839)	0.00015472	TDB
period	620.6349914401015	0.076313	d
	1.70	0.0002089	yr
n	.5800510847199697	7.1323e-05	deg/d
Q	1.841849274657174	0.00015098	au

Orbital elements and their 1-sigma uncertainty. Screenshot from the NASA/JPL Small-Body Database

The parameters are: ***e*** (Eccentricity), ***a*** (Semi-major axis), ***q*** (Perihelion), ***i*** (Inclination), ***node*** (Argument of ascending node), ***peri*** (Argument of periapsis/perihelion), ***M*** (Mean anomaly), ***t_p*** (Epoch in JD at the time of the perihelion passage), ***period*** (Period around the Sun), ***n*** (Mean motion) and ***Q*** (Aphelion). Please note that the epoch corresponds to a mean anomaly of 0 degrees (passage at the perihelion).

Let's set all necessary parameters as constants:

The database table provides 1-sigma uncertainties for the orbit determination. *Thus, we simply re-sample a set of elements (based on the uncertainty) and compute several positional vectors, right?*

Not exactly. The shown parameters are correlated! For example: A large eccentricity does not necessarily correspond to a large perihelion. What we need is the so called ***covariance matrix*** of the orbital elements, to determine the linked uncertainties.

Orbit Covariance (6x6)						
	e	q	tp	node	peri	i
e	3.514700707269822E-9	-1.133056768967312E-10	-8.77949537340269E-9	-5.609507305859223E-9	8.765173957949147E-9	3.251989501493864E-8
q	-1.133056768967312E-10	3.658258147221794E-12	2.792852271630767E-10	1.7929510511293E-10	-2.846448772903082E-10	-1.048525398906457E-9
tp	-8.77949537340269E-9	2.792852271630767E-10	2.469861185148112E-8	1.523164306525229E-8	-2.047449469529637E-8	-8.110945197319675E-8
node	-5.609507305859223E-9	1.7929510511293E-10	1.523164306525229E-8	1.000636233259793E-8	-1.389698158070334E-8	-5.18922599787702E-8
peri	8.765173957949147E-9	-2.846448772903082E-10	-2.047449469529637E-8	-1.389698158070334E-8	2.314557673578479E-8	8.120674279383214E-8
i	3.251989501493864E-8	-1.048525398906457E-9	-8.110945197319675E-8	-5.18922599787702E-8	8.120674279383214E-8	3.009070096457002E-7

Covariance matrix of some orbital elements (considering the same dimensionality: AU, degrees, etc.). Screenshot from the NASA/JPL Small-Body Database

We hard-code the orbital elements in the following coding part, however, we only consider the first 2 significant digits. Please note: According to the PEP8 coding guidelines, only one white-space is allowed after a comma. For better readability I purposely violate this rule:

Part 3/12

Now we can use *numpy*'s function multivariate_normal to compute a sample set of orbital elements, based on the provided mean values and the covariance matrix. We re-sample 1000 sub-sets.

Part 4/12

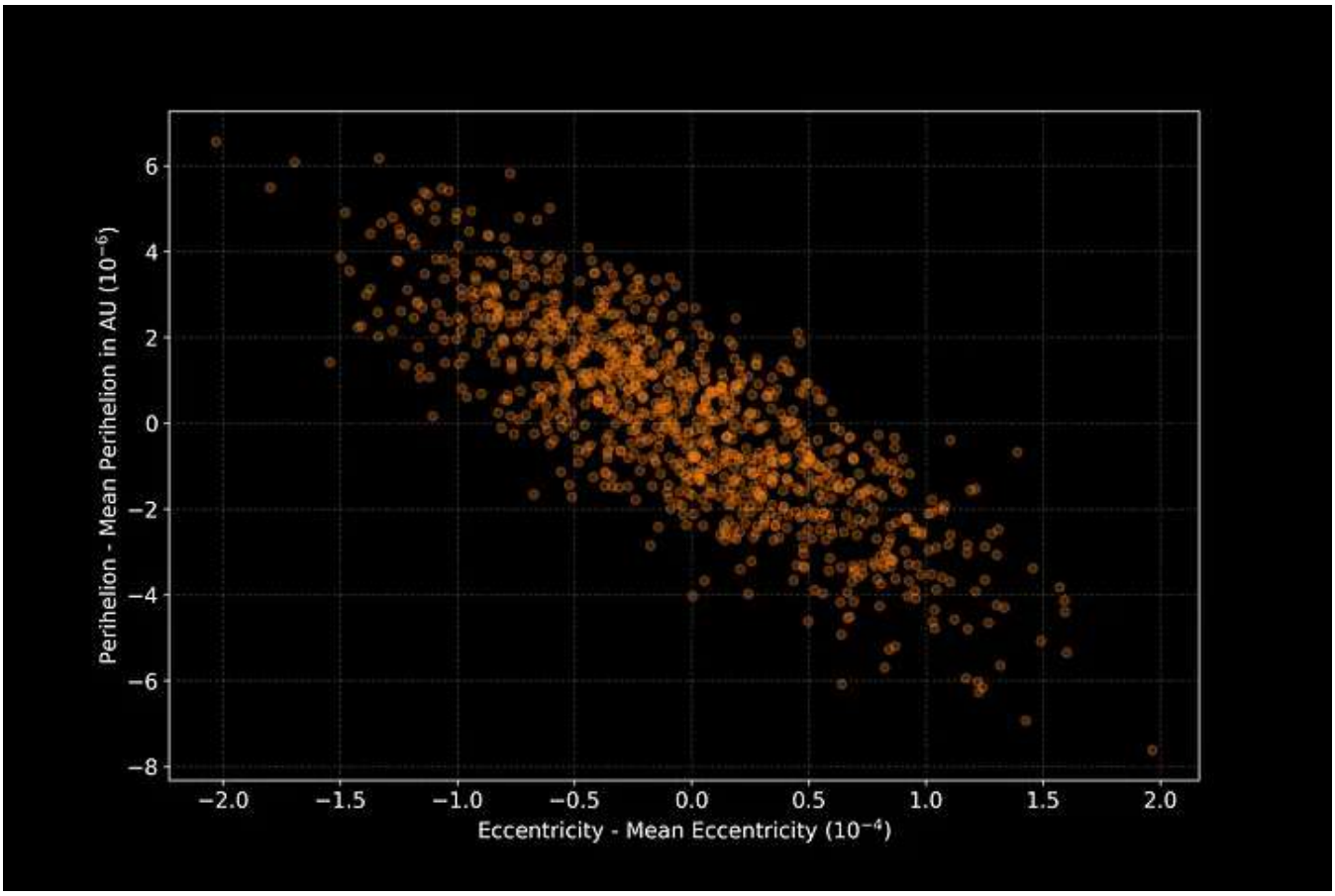
Before we continue to work with the solution space of 1000 orbital element sets, the data are moved into *pandas* dataframes. The order of the elements corresponds to the column/row order of the covariance matrix. E.g., line 6 assigns the eccentricity column from the 0th column. In line 9 to 10, the SPICE function convrt is used to convert the perihelion values, given in AU, to km. In line 16 to 18 the epoch that is

given in Julian Date (JD) is converted in Ephemeris time (ET) using utc2et. Finally, all angular values are converted in radians (line 22 to 24) and the last column stores the Sun's G*M value.

Part 5/12

Since the covariance matrix is not a unit matrix, orbital elements correlate. The following code snippet creates a scatter plot where the perihelion is plotted vs. the eccentricity (line 15 to 17). The values are shifted by the corresponding mean values and scaled for better readability. Below you can see the resulting figure.

Uncorrelated values would result in a spherical-like distribution. Here, the scatter plot shows clearly that larger eccentricities correlate with smaller perihelia. If we would have used the 1-sigma uncertainty for each orbital element independently, the resulting distribution would be completely biased and would not represent the actual solution space.



Shifted and scaled perihelion vs. eccentricity of the re-sampled asteroid data. Credit: T. Albin

In the next step, we can finally compute state vectors of the asteroid for each re-sampled orbital element set. The date-time of interest: The Asteroid Day (line 2). We set the corresponding ET in line 5 and add the 0 degrees mean anomaly in line 6. Line 9 to 19 use now the SPICE function conics to determine the state vector for each re-sampled set. The resulting state vectors provide the x,y and z positional information and the corresponding velocities; given in km and km/s, respectively. Since the orbital elements are provided with respect to the Sun, the origin of the resulting state vectors is our Solar System's star. The positional vectors are extracted in line 22 and 23.

Part 7/12

We have now 1000 positional vectors. *What is the largest distance of all distance combinations?* Let's find out (task for you: you can create a Kernel-Density Estimator plot of all distances to derive more statistical insights). We use the *scipy* function cdist. The distance computation is performed in line 5 and 6. The maximum distance is then printed at the end in an f-string.

Part 8/12

The resulting maximum positional variation is 1800 km in space. 2020 JX1 passed our home planet at around 3 Lunar Distances (1 LD is approximately 300,000 km);

thus, 1800 km is not a lot. However, let's check the resulting ecliptic longitude and latitude values of the solution space as seen from Earth.

```
Maximum distance of the predicted positions of 2020 JX1 in km:
1824.7702017441325
```

For this purpose, we need the positional vector of our home planet as seen from the Sun. We apply the SPICE function spkgps to determine the x, y and z components in km ...

Part 9/12

... and apply a vector addition to transform the Sun-centric asteroid vectors to Earth-centric ones:

Part 10/12

Finally, we can compute the ecliptic longitude and latitude values by using the SPICE function recrad (line 2 to 4 and 5 to 7). Line 10 to 14 print some general statistics of the values.

The minimum and maximum ecliptic longitude values are 109.43 and 109.47 degrees, respectively. The latitude variation is between 26.96 and 26.98 degrees. Both coordinates do not vary a lot and it appears that the covariance matrix of the asteroid is good enough to properly track the asteroid during the close encounter with Earth!

```
Statistics for the Ecliptic Longitude:
count    1000.000000
mean      109.446769
std         0.005369
min       109.431791
25%       109.443078
50%       109.446638
75%       109.450644
max       109.466238
Name: ECLIP_LONG_DEG, dtype: float64

Statistics for the Ecliptic Latitude:
count    1000.000000
mean       26.973368
std         0.003825
min        26.962614
25%        26.970686
50%        26.973686
75%        26.975981
max        26.982502
Name: ECLIP_LAT_DEG, dtype: float64
```

Conclusion

Computing the state state vector of minor bodies is not as trivial as it appears in the beginning of the tutorial sessions:

1. Gravitational perturbations alter the trajectory all the time
2. Measurement errors lead to orbital elements with uncertainties

We have several possibilities to handle this: with sophisticated modelling and continuous surveys and follow up observations. Every day we discover more and more asteroids, and some are potentially hazardous. We cannot simply *fill a*

database and leave it there. We need to actively work on it to discover and identify cosmic hazards as soon as possible.

In the meantime: Have a nice *Asteroid Day 2020*

Thomas

WATCH

A global awareness movement to protect Earth from asteroid impacts

asteroidday.org

Assignment

The last coding block in the script provides 2 tasks for you. Have fun!

Part 12/12

Space Science Python Space Science With Python Data Science

Sign up for The Variable

By Towards Data Science

Every Thursday, the Variable delivers the very best of Towards Data Science: from hands-on tutorials and cutting-edge research to original features you don't want to miss. Take a look.

By signing up, you will create a Medium account if you don't already have one. Review our Privacy Policy for more information about our privacy practices.

Get this newsletter

About Help Terms Privacy

Get the Medium app

Download on the App Store
GET IT ON Google Play